

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Codebase Documentation

Repository map, internals of every module, and extension recipes

Project	System Scanner Pro (v3.1)
Document	Codebase Documentation
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. How to Read This Volume	4
2. Repository Layout at a Glance	4
3. Root-Level Files Reference	4
4. Entry Point: admin/main.py	5
5. Entry Point: api/index.py (Vercel)	5
6. Entry Point: client/main.py	5
7. The django_admin Project Package	5
8. settings.py - Core Configuration	6
9. settings.py - Security & Auth Backends	6
10. settings.py - Database Strategies	6
11. settings.py - Channels, REST & CORS	6
12. settings.py - Logging & Static	7
13. URL Configuration Map	7
14. Middleware Chain Internals	7
15. App scanner_api - Overview & File Map	7
16. scanner_api Models - Tenancy & Identity	8
17. scanner_api Models - Clients & Scans	8
18. scanner_api Models - Org Structure	8
19. scanner_api Models - Asset Management	8
20. scanner_api Models - Security & Audit	9
21. Views - Authentication Cluster	10
22. Views - Client Lifecycle Cluster	10
23. Views - Scan & Monitoring Bridges	10
24. JWT Internals (jwt_auth.py)	10
25. API Keys Internals (api_key_auth.py)	10
26. Cookie Fallback Internals (middleware.py)	11
27. Agent HMAC Module (monitoring/security.py)	11
28. Supabase Integration (supabase_client.py)	11
29. App monitoring - Overview & Engines	11
30. WebSocket Consumers Internals	12
31. Scheduler & Management Commands	12
32. App maintenance - File Map & Features	12
33. App intelligence - File Map & Report Engine	12
34. Frontend - Templates & JS Modules	12

35. Tests & Quality Assets	13
36. Extension Guide - Where to Add What	13

1. How to Read This Volume

This is the developer's map of the repository. Section 2 orients you; sections on each Django app explain models, views and support modules; client-agent sections mirror the agent's real file layout (main.py plus a dedicated events/package); closing sections cover tests, frontend assets and where to add new features. Line references cite the state of the code at documentation time and may drift as the project evolves.

2. Repository Layout at a Glance

```
AdminClient/
|- admin/                Django server
|   |- manage.py
|   |- main.py           server launcher (HTTP/ASGI/UDP discovery)
|   |- scanner.py        local scan helpers for self-monitoring
|   |- django_admin/     project package (settings, urls, asgi, wsgi)
|   |- scanner_api/      core app: auth, clients, scans, org, assets
|   |   |- views.py      (~4,100 lines - all REST endpoints)
|   |   |- models.py     Company...AssetDocument + ApiKey
|   |   |- urls.py, middleware.py, jwt_auth.py, api_key_auth.py
|   |   |- permissions.py, validators.py, supabase_client.py
|   |   |- scanner.py    admin self-scan helpers
|   |   '- tests/        4 unit test modules
|   |- monitoring/       telemetry app
|   |   |- consumers.py  AgentConsumer / DashboardConsumer (WS)
|   |   |- scheduler.py  APScheduler wiring
|   |   |- security.py   HMAC + rate limiter for agents
|   |   |- anomaly_detection.py predictive.py health.py
|   |   |- feature_store.py diff_utils.py event_bus.py
|   |   |- models.py urls.py management/commands/
|   |   '- static/js/monitoring.js
|   |- maintenance/     records, warranty, downtime, licenses
|   '- intelligence/     alerts, rules, notifications, reports
|- templates/ (30 page templates) static/js/ (15 ES modules)
|- client/              agent source (see section 30)
|- api/index.py          Vercel WSGI entry
|- build_client.py        PyInstaller packaging
|- deploy/{install.sh, nginx/, systemd/}
|- docs/                 prior PDF + its generator script
|- setup_new_supabase.sql setup_cloud_discovery.sql
'- README.md requirements.txt runtime.txt vercel.json ...
```

3. Root-Level Files Reference

File	Purpose
README.md	Authoritative overview: architecture summary, quick start, feature list
requirements.txt	Server dependencies with version ranges (see volume 01 table)
runtime.txt	python-3.11 runtime pin consumed by Vercel
vercel.json	Routes all paths to api/index.py; 30 s function cap
.env.template	Environment variable documentation - CONTAINS LIVE-LOOKING VALUES; rotate and replace with placeholders
.gitignore	Ignores .env*, venvs, __pycache__; note that .env.template slipped through
MANUAL_TESTING.txt / .xlsx	30 essential manual test cases (from 100), owned per team member
VERCEL_FIX.md	Deployment troubleshooting notes for the serverless path
setup_new_supabase.sql	Creates cloud schema incl. server_registry discovery row
setup_cloud_discovery.sql	Resets/publishes the discovery row for a new host
build_client.py	Builds dist/client_scanner.exe via PyInstaller
.github/workflows/supabase-register.yml	Hourly Action publishing ADMIN_SERVER_URL into server_registry

Table 1 - Root files

4. Entry Point: admin/main.py

A single launcher that makes local and VPS operation one command. Responsibilities, in order: parse flags (--port, --host, --asgi, --domain, --signed-cookies, --no-open), apply migrations programmatically, seed a default superuser when the user table is empty, start the monitoring scheduler, register the ADMIN self-client (an always-online Client row used to demo dashboards), open a UDP 45000 discovery listener that answers agent broadcasts, then serve via runserver-style HTTP or Daphne ASGI. The --domain flag publishes the public URL into Supabase server_registry when cloud env vars are set.

5. Entry Point: api/index.py (Vercel)

Wraps the Django WSGI application with gunicorn for Vercel's Python builder, forces production settings (DEBUG off, signed-cookie sessions), runs whitenoise for static files, and exposes a guarded POST /__reset/ maintenance endpoint (token from RESET_TOKEN env; override the hardcoded fallback) that truncates application tables for demo resets. WebSocket consumers are not reachable through this WSGI path by design.

6. Entry Point: client/main.py

Agent orchestrator (~1,150 lines): parses CLI flags (--once, --silent, --rescue, --uninstall), enforces a single-instance mutex, registers OS autostart (Windows Registry Run key / Startup folder), resolves the server through config -> discovery, registers with key_manager + fingerprint identity, starts communicator threads (WS + heartbeat every 30 s), spawns event monitors via events/dispatcher, and coordinates scans triggered locally, on schedule, or pushed by scan_now over WebSocket.

7. The django_admin Project Package

File	Contents
settings.py	All configuration (sections 8-12)
urls.py	Root URLconf: includes scanner_api at root and under /api/, mounts admin site
asgi.py	ASGI entry with channels routing wrapped for Daphne
wsgi.py	Classic WSGI entry used by gunicorn/api-index path

Table 2 - Project package files

8. settings.py - Core Configuration

- `BASE_DIR` points at `admin/`; `SECRET_KEY` reads `DJANGO_SECRET_KEY` with an insecure fallback (flagged in hardening checklists).
- `DEBUG` defaults `True` unless `DJANGO_DEBUG=False`.
- `ALLOWED_HOSTS` from `DJANGO_ALLOWED_HOSTS`, defaulting to `['*']`.
- `INSTALLED_APPS`: `django` contribs + `rest_framework` + `corsheaders` + `channels` + the four project apps (`scanner_api`, `monitoring`, `maintenance`, `intelligence`) + `whitenoise.runserver_nostatic`.
- `Templates`: `APP_DIRS` with the `admin/templates` directory context processors standard set.

9. settings.py - Security & Auth Backends

- Password validators include the custom `validators.validate_strong_password` (10+ chars, upper, lower, digit, special).
- `AuthenticationMiddleware` standard; `CookieAuthMiddleware` appended last so the signed `scanner_auth` cookie can restore sessions when DB sessions are unavailable.
- `SECURE_BROWSER_XSS_FILTER` style headers are additionally forced by `SecurityHeadersMiddleware`.
- Session engine switches to `signed_cookies` on serverless; signed cookies also available via `--signed-cookies` locally.

10. settings.py - Database Strategies

Condition	Resulting engine
<code>DATABASE_URL</code> unset (default)	SQLite at <code>admin/data/scanner.db</code> ; when running frozen (PyInstaller), the DB migrates to a persistent OS app-data directory (unit-tested legacy copy logic)
<code>DATABASE_URL</code> set + serverless detected	PostgreSQL via <code>dj-database-url</code> ; connection rewritten to Supabase pooler transaction port 6543, <code>sslmode=require</code> , <code>CONN_MAX_AGE=0</code>
<code>DATABASE_URL</code> set (persistent host)	PostgreSQL direct with <code>DB_CONN_MAX_AGE</code> honoured

Table 3 - Database selection rules

11. settings.py - Channels, REST & CORS

- `ASGI_APPLICATION` = `django_admin.asgi.application`; channel layer = `channels.layers.InMemoryChannelLayer` (both deployment branches - swap for Redis to scale out).
- `REST_FRAMEWORK`: `CsrfExemptSessionAuthentication` replaces the default session auth (API clients are cookie/session or JWT based); `DEFAULT_PERMISSION_CLASSES` intentionally empty so views opt in explicitly.
- `CORS_ALLOW_ALL_ORIGINS` = `True` - acceptable for internal demos, tighten for production.

12. settings.py - Logging & Static

- LOGGING: console handler INFO; named loggers scanner_api, monitoring, channels, daphne.
- Server launcher additionally tees output into admin/local_server.log / .err.
- STATIC_URL /static/ with whitenoise compression in production paths; CDN links for Bootstrap/Icons/Chart.js live in templates/base.html.

13. URL Configuration Map

Prefix	Module	Highlights
(root pages)	scanner_api.urls	/ login/ signup/ logout/ dashboard /connect/<user>/<company>/ /download-client/ /client/<key>/ /settings/ /admin-server/ /admin-page/ /executive-dashboard/ /account/ /scans/ /audit-log/
(org & assets)	scanner_api.urls	/employees/ /departments/ /locations/ /assets/ /assets/<uuid>/ /asset-dashboard/
/api/	scanner_api.urls	~90 JSON endpoints: auth/*, token/*, register, ping, submit-scan, clients, groups, settings, apikeys, employees/departments/locations/assets CRUD, import/export
/monitoring/	monitoring.urls	dashboard page + agent API (register-agent, heartbeat, metrics, events, inventories, schedules) + devices API
/maintenance/	maintenance.urls	records/history/documents/warranty/downtime pages+API, licenses, compliance, alerts
/intelligence/	intelligence.urls	alerts/rules/notifications/reports/scheduled-reports/compliance/audit pages + APIs
/ws/	monitoring.routing	ws/agent/<agent_id>/ (HMAC) and ws/dashboard/ (session)
/django-admin/	django.contrib.admin	Standard admin for superusers

Table 4 - URL surface

14. Middleware Chain Internals

Outermost to innermost: SecurityHeadersMiddleware (adds X-Content-Type-Options nosniff, X-Frame-Options DENY, X-XSS-Protection, Referrer-Policy strict-origin-when-cross-origin) -> WhiteNoise -> Session -> Common -> CsrfView -> Authentication -> Message -> XFrameOptions -> CookieAuthMiddleware. CookieAuthMiddleware (scanner_api/middleware.py) reads the scanner_auth cookie - value user_id signed by TimestampSigner(salt='scanner-auth-cookie', max_age 30 days) - and attaches request.user before view dispatch, which keeps dashboards functional under signed-cookie sessions.

15. App scanner_api - Overview & File Map

File	Role
models.py	25 models spanning tenancy, clients, scans, org, assets, audit (sections 16-20)
views.py	~4,100 lines; every REST endpoint + page views + helper functions (get_user_company, ensure_admin_client, admin_self_scan...)
urls.py	Page routes + /api/ router shown in section 13
middleware.py	CookieAuthMiddleware described above
jwt_auth.py	Issue/verify HS256 pair (section 24)
api_key_auth.py	ApiKey model + verification + rate limiting (section 25)
permissions.py	IsSuperAdmin / IsViewer / CanViewReports opt-in classes
validators.py	Strong password validation
supabase_client.py	Thin REST wrapper for discovery registry (section 28)
scanner.py	Hostname/platform helpers reused for the ADMIN self-client
tests/	Registration/approval, auth redirects, cookie fallback, persistent DB

Table 5 - scanner_api files

16. scanner_api Models - Tenancy & Identity

Model	Key fields	Notes
Company	name, slug	Tenant root; everything company-scoped hangs off it
AdministratorProfile	user 1-1, company FK, contact fields	Extends auth_user; mfa_enabled/mfa_secret fields exist but no TOTP flow yet

Table 6 - Tenancy models

17. scanner_api Models - Clients & Scans

Model	Key fields	Notes
ClientGroup	name, company FK	Grouping for dashboards/policies
Client	registration_key UK, hostname, platform, status(pending/online/offline), approved, auto_approved, owner FK, company FK, group FK, tags, purchase info, scan_interval, scan_enabled, scan_requested, last_ip, device_fingerprint, deleted, client_version, os_version, cpu_model, ram_info	Central device record; soft delete via deleted flag; approval lifecycle unit-tested
ScanResult	client FK, scan_type, scan_data JSON, created_at	Raw payload preserved; indexed by client+time
AddonDevice	client FK, name, description, type, purchase info	Peripherals attached to a machine

Table 7 - Client/scan models

18. scanner_api Models - Org Structure

- Location (name/address/company) -> Department (location FK) -> Employee (emp_id, name, email, designation, department FK).
- EmployeeAssetAssignment links employees to Assets with assignment metadata.
- OrgAuditLog captures structural changes for traceability.

19. scanner_api Models - Asset Management

Model	Purpose
AssetCategory / AssetVendor	Cataloguing masters for assets
Asset	UUID PK, asset_tag, serial, status state machine Draft->Pending->Approved->Purchased->Assigned->Retired (guarded transitions), purchase cost/date/vendor, warranty expiry, QR-friendly fields
AssetAssignment	Assignment v2 rows (asset, employee, dates, status, notes)
AssetTransfer / AssetHistory	Transfers between custodians; immutable history trail
AssetDocument	Base64-stored documents (invoices, warranties) attached to assets

Table 8 - Asset models

20. scanner_api Models - Security & Audit

Model	Purpose
ActivityLog	Company-scoped action feed (login/update/delete notes)
AuditLog	Actor/action/object audit entries
LoginHistory / LoginAttempt	Successful logins with IP/user-agent; raw attempt stream feeding logout
DeviceFingerprint	Historical fingerprints seen for a registration key
ApiKey	sk_-prefixed keys stored SHA-256 hashed; name, scopes, rate limits, IP allowlist, last_used
Setting	Key/value store (password reset tokens, admin_client_key, toggles)

Table 9 - Security models

21. Views - Authentication Cluster

View (views.py)	Behaviour highlights
AuthLoginView	Lockout window (5 fails / 30 min) enforced via LoginAttempt; writes LoginHistory + ActivityLog; sets session + signed cookie
AuthLogoutView / AuthMeView	Session teardown; current identity probe for the SPA-ish JS
ChangePasswordView	Validates old password + strength policy; audits change
ForgotPasswordRequestView	Generates TimestampSigner token (salt 'scanner-password-reset', 1 h), stores in Setting; responds success but SENDS NO EMAIL (documented gap)
ForgotPasswordResetView	Unsigns token, cross-checks stored copy, applies new password, clears token
SupabaseRegisterView	Receives registry callbacks / publishes server URL when cloud mode active

Table 10 - Authentication endpoints

22. Views - Client Lifecycle Cluster

RegisterClientView - creates/updates Client rows; demotion rules: different fingerprint resets approval, never-approved keys stay Pending, blank hostname falls back to machine name.

PingClientView - refreshes last_seen/status; demotes stale auto-approvals when auto-approve is off (auto-approve itself is hardcoded False).

Approve/reject/delete endpoints - administrator actions recorded in ActivityLog; approving a soft-deleted client reactivates it.

ensure_admin_client() - guarantees an ADMIN-<HOSTNAME> Client row exists (stable key persisted in Setting) so dashboards always show the server itself.

23. Views - Scan & Monitoring Bridges

TriggerScanView - sets scan_requested=True and pushes scan_now over the agent WebSocket group when connected; falls back to flag polling otherwise.

ScanAllView - fans out to every enabled client; per-client results logged.

LocalScanView - runs the admin server's own scanner suite synchronously for demos.

SubmitScanView - accepts agent payloads: stores ScanResult JSON, upserts normalised inventory via monitoring models, recomputes health score, evaluates diff + anomaly pipelines.

24. JWT Internals (jwt_auth.py)

- Algorithm HS256; signing secret = DJANGO_SECRET_KEY (rotate carefully - invalidates tokens).
- Access tokens: 60 minutes. Refresh tokens: 7 days.
- POST /api/token/ issues a pair after credential check; POST /api/token/refresh/ rotates access.
- Claims carry user id + username; verification rejects expired/bad signatures with 401.

25. API Keys Internals (api_key_auth.py)

- Keys render once as sk_; only SHA-256 hash persisted with a lookup prefix.
- Verification middleware/helper resolves key -> ApiKey row, checks active flag, IP allowlist, and sliding-window rate limit counters before attaching identity.
- last_used timestamp updated on success; revocation = deactive flag (instant).

26. Cookie Fallback Internals (middleware.py)

Signed cookie format: `TimestampSigner(salt='scanner-auth-cookie').sign(str(user_id))` with `max_age 30 days`, cookie name `scanner_auth`, `HttpOnly`. On requests without a Django session - typical on serverless cold starts - the middleware unsigns the value and lazily attaches the user. Tampering fails signature validation; expiry returns anonymous.

27. Agent HMAC Module (monitoring/security.py)

generate_api_secret() - `secrets.token_hex(32)` - 64-char hex `AgentSecret` rows per agent.

compute_signature/verify_signature - `HMAC-SHA256(secret, raw body)` compared with `hmac.compare_digest`.

verify_timestamp - X-Timestamp within ± 300 s blocks replay.

authenticate_agent(request) - orchestrates header extraction (X-Agent-ID/X-Signature/X-Timestamp), lookup, verification, `last_used` update.

RateLimiter - in-memory fixed-window limiter keyed by agent/IP (60 req/min default).

28. Supabase Integration (supabase_client.py)

Small REST client using `SUPABASE_URL` + `SUPABASE_SERVICE_KEY` to upsert/read the single-row `server_registry` table (`id='admin'`): `ip_address/port/protocol/server_name/is_active`. The hourly GitHub Action duplicates the publish path as a safety net using repo variables. Agents read this row during discovery when UDP broadcast fails (e.g. across subnets).

29. App monitoring - Overview & Engines

Module	What it does
<code>models.py</code>	<code>DeviceMonitoringInfo</code> (1-1 health snapshot), <code>HardwareInventory</code> , <code>SoftwareInventory</code> , <code>DeviceHeartbeat</code> , <code>DeviceMetrics</code> , <code>DeviceHistory</code> , <code>DeviceAlert</code> , <code>AgentVersion</code> , <code>AgentSecret</code> , <code>ScheduledScan</code> , <code>ScanScheduleLog</code> , <code>PendingScan</code>
<code>health.py</code>	Weighted 0-100 score across CPU/RAM/disk/uptime signals with component breakdown
<code>anomaly_detection.py</code>	Methods: <code>zscore</code> , <code>iqr</code> , <code>moving_avg</code> , <code>threshold</code> , <code>compound</code> ; per-metric sensitivity tuning
<code>predictive.py</code>	Disk-full ETA via linear regression on usage trend; failure risk 0-100; capacity forecast horizons
<code>feature_store.py</code>	Extracts 20+ ML-ready features per device (means, slopes, volatility) for future modelling
<code>diff_utils.py</code>	<code>compute_scan_diff()</code> -> added/removed/changed hardware & software between consecutive scans
<code>event_bus.py</code>	Lightweight pub/sub connecting ingest to analytics/alert listeners

Table 11 - Monitoring engines

30. WebSocket Consumers Internals

AgentConsumer (/ws/agent/<agent_id>) - handshake validates query-param signature/timestamp against AgentSecret (HMAC), accepts, joins group device_<id>; handles heartbeat_json, scan_now command relay, ack/result frames; updates DeviceHeartbeat/MonitoringInfo.

DashboardConsumer (/ws/dashboard/) - session-authenticated browser socket subscribing to status/alert broadcasts; pushes live card updates consumed by static/js/websocket.js.

Routing - monitoring/routing maps both paths; InMemory layer means single-process fan-out (see trade-offs, volume 02).

31. Scheduler & Management Commands

Job / command	Interval / trigger	Effect
stale_checker	60 s	Marks heartbeats stale; feeds offline_detector
offline_detector	120 s	Flips silent devices to offline; raises alerts if configured
alert_checker	300 s	Evaluates alert rules against latest metrics/inventory
health_checker	300 s	Recomputes health scores; triggers predictive checks
scan_local	manual	Scans the server host itself into its ADMIN client row
scan_all	manual	Requests scans from all enabled agents
run_alert_checks	manual	One-shot rule evaluation (useful after imports)
clear_data	manual	Danger: wipes operational data (demo reset)

Table 12 - Background work catalogue

32. App maintenance - File Map & Features

- models.py: MaintenanceRecord (+approval workflow), MaintenanceHistory, MaintenanceDocument, WarrantyRecord, DowntimeRecord, SoftwareLicense (masked key display + encrypted full key field), LicenseAssignment, LicenseHistory, ComplianceRecord, MaintenanceAlert.
- views.py + urls.py: page + REST CRUD for each entity; serializers.py shapes API payloads.
- alerts.py: generates MaintenanceAlert rows for upcoming service dates, warranty expiries, license renewals and prolonged downtimes.
- templates.py: inline template fragments used by list/detail pages.

33. App intelligence - File Map & Report Engine

- models.py: Alert(+AlertHistory), AlertRule, Notification(+NotificationPreference), Report, ScheduledReport, AuditLogEntry, ComplianceLog, DashboardAnalytics snapshots, RetentionPolicy.
- reports.py: report_engine building ~25 document types (asset inventory, device health, license compliance, audit trails, compliance packs...) exporting PDF (ReportLab), Excel (openpyxl) and CSV; ScheduledReport rows drive recurring generation.
- alerts.py: unified alert ingestion bridging monitoring DeviceAlerts and rule evaluations.
- notifications.py: fan-out to Notification rows honouring per-user preferences.
- audit.py: structured AuditLogEntry writer used across apps for sensitive actions.

34. Frontend - Templates & JS Modules

Layer	Contents
templates/base.html	Dark-themed shell: sidebar nav, Bootstrap 5.3.3, Icons 1.11.3, Chart.js 4.4.0 (CDN), toast + modal scaffolding
Page templates (29 more)	dashboard, client_detail, scan_history, settings, admin_server/admin_page, executive_dashboard, account, audit_log, assets trio, employees/departments/locations, monitoring, maintenance/licenses/maintenance_alerts, intelligence suite (alerts/reports/scheduled_reports/notifications/compliance/audit_logs/dashboard), connect, signup, login
static/js/app.js	Shared helpers: fetch wrappers, toasts, formatting, nav state
static/js/websocket.js	DashboardConsumer client: reconnect/backoff, event dispatch to page handlers
Page modules	dashboard.js, detail.js, scan_history.js, settings.js, account.js, admin_page.js, executive-dashboard.js, assets.js, asset_dashboard.js, asset_detail.js, employees.js, departments.js, locations.js + monitoring/static/js/monitoring.js

Table 13 - Frontend inventory

35. Tests & Quality Assets

Artefact	Coverage
tests/test_registration_approval.py	13 scenarios: pending-by-default, same/different fingerprint handling, ping demotion, approval persistence, connect-URL company moves, deleted-client reactivation, admin seed
tests/test_cookie_auth_fallback.py	Signed cookie authenticates user; works after AuthenticationMiddleware ordering
tests/test_persistent_database.py	Legacy SQLite copied into persistent data dir on frozen runs
tests/test_auth_redirects.py	next-path prefixing, empty-next fallback, login-page redirects, robustness
MANUAL_TESTING.txt/.xlsx	30 essential cases across 17 modules with owner assignments - ALL marked Pending (see volume 08)
CI	None for tests; only supabase-register.yml keep-alive workflow exists

Table 14 - Quality artefacts

36. Extension Guide - Where to Add What

I want to add...	Touch these files
A new REST endpoint	scanner_api/views.py + urls.py (+ serializer if model-backed); reuse get_user_company scoping
A new model	Appropriate app models.py -> makemigrations -> migrate; expose via views if needed
A new scheduled job	monitoring/scheduler.py job list (or management command for manual runs)
A new WS message type	monitoring/consumers.py receive branch + client/communicator.py sender + dashboard JS handler
A new agent collector	client/scanner.py (inventory) or client/metrics.py (telemetry); payload merges automatically
A new event monitor	client/events/<name>_monitor.py mirroring usb_monitor.py; register in dispatcher
A new report type	intelligence/models.py REPORT_TYPE_CHOICES + intelligence/reports.py builder
A new page	template in admin/templates + JS module + URL entry + nav link in base.html

Table 15 - Extension cookbook